Technical walkthrough: everything we did, step by step

Hybrid post-quantum TLS on the StarFive VisionFive 2 (JH7110, 4 × SiFive U74 RV64GC, 1.5 GHz, Debian). Detailed reports and raw data for every step: `achieved_results/`. Practice questions: `achieved_results/study_guide/`.

The goal

The task asked for three things: - Hybrid key exchange (ML-KEM + a classical algorithm). - Hybrid authentication (ML-DSA + ECDSA, replacing RSA). - **No increase in end-to-end latency** compared with the legacy baseline: X25519 + RSA-3072, the original connection.

Stage 1: Getting the board ready

- **Access:** the board has no SSH, so it was controlled from a Mac over the **USB serial console** (115200 baud). It reaches the internet through the Mac's Internet Sharing (board 192.168.2.2, Mac 192.168.2.1). Files went across with a small web server on the Mac and `wget` on the board.
- **The library problem:** Debian on the board ships **OpenSSL 3.0.9**, which has no ML-KEM or ML-DSA.
- **Fix:** we built **OpenSSL 3.5.9** from source on the board (about 30–40 minutes). 3.5 is the long-term-support release with both algorithms built in, in their final FIPS 203/204 form.
- It's installed in a separate folder (`/opt/openssl-3.5`), and our programs are linked to it directly, so the system's own OpenSSL is untouched and nothing else on the board breaks.
- **Why not liboqs/oqs-provider or another library?** Native OpenSSL 3.5 has fewer moving parts and is what a real product would ship. Python's `ssl` module was ruled out because it's tied to the system OpenSSL 3.0.

Stage 2: Hybrid key exchange

- **`pqc_client`** (C, on the board) connects to the hackathon server and asks for the TLS group **`X25519MLKEM768`**.
- **How it works:**
- The client sends an X25519 public key plus an ML-KEM-768 public key (1,184 B).
- The server replies with its X25519 key plus an ML-KEM ciphertext (1,088 B).
- Both shared secrets are combined into the TLS key schedule, so the session is safe if **either** algorithm holds.
- **Proof it worked:** the server replied `"handshake_method":"hybrid-pqc"`.
- **An early mistake we caught:** the first legacy result of 165 ms was wrong, because a local relay program added its own connection time. Removing it gave the real numbers. That's why all later tests used large, alternating runs.

Stage 3: Hybrid authentication

The problem: the hackathon server only signs with RSA-3072, so hybrid authentication can't be tested against it.

The solution: our own TLS server, **`hybrid_server`** (C, running on the Mac). It holds: - an **RSA-3072 certificate**, for legacy clients (the baseline), and - an **ECDSA P-256 certificate plus ML-DSA keys**, for hybrid clients.

Both modes are measured against the same server, so the comparison is fair. The matching client is **`hybrid_client`** (C, on the board); it measures TCP, handshake and end-to-end time, plus bytes and memory.

Version 1 (dual signature after the handshake): 1. A normal TLS 1.3 handshake runs, and the server proves its identity with **ECDSA**. 2. The server then signs `"hybrid-auth-v1"` + a **TLS exporter value** with **ML-DSA**. The exporter is a 32-byte secret unique to this connection (`SSL_export_keying_material`), so the signature can't be replayed in another session. 3. The signature travels inside the first HTTP response, so there's **no extra round trip**. 4. The client accepts only if **both** signatures verify.

Trust: the client **pins** the server certificate and the ML-DSA public key, so it trusts exactly that server and no certificate authority. In OpenSSL terms: `SSL_VERIFY_PEER`, the certificate as the only trust anchor (`X509_V_FLAG_PARTIAL_CHAIN`), and a check of the IP address.

Security tests, all passed: - A wrong ML-DSA key is rejected. - A wrong certificate is rejected. - (For v2) a missing signature is rejected.

Stage 4: First optimisations, at full CPU speed (day 1)

At first, hybrid was +3.6 ms slower than legacy. We found the reasons and fixed them one by one:

Step	Why	Result
Profiled the signatures (`sigbench`)	Find where the time goes	**ECDSA verify took 4.1 ms**: the bottleneck was the classical part, not the PQ part
Rebuilt OpenSSL with **`enable-ec_nistp_64_gcc_128`**	The default build uses slow generic P-256 code on RISC-V	ECDSA verify **4.1 → 1.4 ms** (2.9× faster)
ML-DSA-65 → ML-DSA-44	Category 2 matches P-256's strength (as in the IETF composite draft)	Smaller signature (2,420 vs 3,309 B), faster verify
Binary signature instead of base64	Base64 adds 33 %	About 800 B saved
CPU governor **`performance`** (fixed 1.5 GHz)	The default clock was slow (see stage 5)	Hybrid **10.3 ms vs legacy 10.7 ms**: target met

We also learned that **RSA moves the cost to the server**: RSA-3072 signing takes 34 ms on the board, while ECDSA + ML-DSA-44 signing takes 5.3 ms. So for devices that sign (mutual TLS), hybrid is much cheaper.

The problem with full speed: the chip ran hot (about 50–52 °C idle). We turned full speed off again and kept those results as a reference.

Stage 5: The proper baseline (default clock, 1000 runs)

- **Method:** 1000 connections per mode, in 20 blocks of 50, alternating which mode goes first. Both modes see the same network and temperature, and we compare block by block.
- **Result at the default setting:** hybrid **27.95 vs 21.61 ms** legacy on our server (+29 %), and +13 % on the hackathon server.
- **Lesson:** day 1's success depended on full speed. At the default clock, the target was missed. Everything after this is about closing that gap **without** full speed.

Stage 6, approach 1: Dynamic CPU frequency

- **Root cause:** the `ondemand` governor idles at **312.5 MHz** and only speeds up after it notices load. A TLS handshake is a ~20 ms burst, mostly over before the clock rises, so the crypto runs at about a fifth of full speed.
- **Idea:** keep the power-saving governor, but raise the **minimum** clock (`/sys/devices/system/cpu/cpufreq/policy0/scaling_min_freq`). One setting, no code change.
- **Screening:** four settings, then a 1000-run confirmation:

Setting	Hybrid	Idle temperature
Default (312.5 MHz minimum)	27.84 ms	44.7 °C
ondemand, minimum 750 MHz	**16.19 ms**	**45.2 °C** ← best balance
ondemand, minimum 1250 MHz	16.67 ms	46.6 °C
schedutil, default minimum	33.85 ms (worse)	46.2 °C
schedutil, minimum 750 MHz	16.54 ms	46.6 °C

- **Result:** hybrid at the 750 MHz minimum (16.19 ms) beats the legacy baseline (21.30 ms in that run), for only +0.5 °C.

Stage 7, approach 2: RISC-V bit manipulation

- **Root cause:** the U74 cores support the **Zba/Zbb** instructions, but normal builds target plain `rv64gc` and don't use them. Without them, a 64-bit rotation takes 3 instructions.
- **Why it matters for PQC:** ML-KEM and ML-DSA spend a lot of their time in **SHA-3/SHAKE (Keccak)**, which is mostly rotations and AND-NOT.
- **Fix:** a third OpenSSL build with `-march=rv64gc_zba_zbb`, with no code changes. The compiler then uses `rol`/`ror`, `andn` and `sh3add`.
- **Micro-benchmarks** (at a fixed 1.5 GHz, because the ondemand clock caused 2× noise):
- SHA-3 **+50 %** faster
- ML-KEM **–13 to –19 %**
- ML-DSA verify **–15 %**
- RSA, ECDSA and X25519 unchanged
- **End-to-end:** hybrid 28.15 → **26.57 ms** (–5.6 %, 20/20 blocks). Small, but free.
- **Catch:** the binary only runs on CPUs with Zba/Zbb.

Stage 8, approach 3: Multicore

Idea: the classical and post-quantum halves are independent, but they ran one after another on one core while three cores sat idle.

Part 1, feasibility test (`parverify.c`): - Three pairs, each timed one after another vs on two cores: ECDSA ‖ ML-DSA verify, X25519 ‖ ML-KEM key generation, X25519 ‖ ML-KEM decapsulation. - Three ways of running them: sequential, a new thread per operation, and a **persistent worker thread**. The persistent worker won, because creating a thread each time costs too much. - 10,800 measurements: **1.5–1.65× faster**. Client crypto went from 16.0 to 10.1 ms at the default clock.

Part 2, into the real TLS handshake (v2): - **The obstacle:** in v1 the ML-DSA signature arrives *after* the handshake, so it can't run in parallel with ECDSA. - **The fix:** the server puts the ML-DSA signature **inside the TLS 1.3 Certificate message**, as a custom extension `0xFE20` (`SSL_CTX_add_custom_ext`). - It signs `"hybrid-auth-v2"` + **SHA-256(ClientHello ‖ ServerHello)**. - The hellos contain both sides' random values and key shares, so the signature is bound to this one connection. - A `msg_callback` captures the hellos. - **On the board:** when the extension arrives, a **persistent worker thread on core 2** verifies ML-DSA, while OpenSSL verifies ECDSA on core 1. The worker uses a pthread with a mutex and condition variable. - **Safety:** before sending any application data, the client waits for the worker. If either check fails, the connection is dropped. - **Result:** hybrid v1 28.19 → v2 parallel **24.57 ms** (–12.8 %, 20/20 blocks), with no extra heat. - **Why not 4 cores:** there are only two independent checks; the rest of the handshake is a fixed sequence (Amdahl's law).

Stage 9: All three combined (final result)

BASE (default setup) and COMBINED (750 MHz minimum + bitmanip + second core) were alternated in one session: **8,000 connections, 0 failures**.

	Legacy baseline	Hybrid, BASE	Hybrid, COMBINED
Own server, full hybrid	21.04 ms	28.06 ms	**14.47 ms (–31 %, 20/20)**
Hackathon server, key exchange	38.36 ms	43.76 ms	**35.62 ms (–7 %, 17/20)**
Idle temperature	45.9 °C	45.9 °C	47.2 °C (+1.3)

Two things to know: - Hybrid gains more than legacy from the same changes (–48 % vs –27 %), because hybrid has more client-side crypto, which is what the fixes speed up. - Like for like on our server, hybrid is even **1 ms faster than legacy**, because RSA signing on the server is slow.

Stage 10: The task-sheet metrics

- **Memory:** 6.3–6.6 MB peak; hybrid adds only +0.1–0.2 MB.
- **Communication:**
- Received 5,105 vs 2,520 B (ML-DSA signature + ML-KEM ciphertext).
- Sent 1,552 vs 330 B (ML-KEM public key).
- About +3.8 KB per handshake in total.
- **Session resumption:** resumed hybrid takes 5.98 ms vs 4.60 ms legacy. There are no signatures, but there's still a fresh hybrid key exchange.
- **Energy:** not measured (no USB-C power meter). Temperature is the proxy.
- **Hailo-8L:** not detected on PCIe. It's an 8-bit neural-network chip, while ML-KEM and ML-DSA need 12- and 23-bit arithmetic, and the PCIe transfer would cost about as much as the work saved. The CPU-side fixes gave more.

Stage 11: Deliverables

- Every approach has its own folder in `achieved_results/` with a report (`.md` + `.pdf`), raw data, statistics, logs and scripts.
- Also in the repository: the combined result, the migration guide (`achieved_results/SUMMARY.md`, section 7), the study guide, and the slide notes (`presentation/TECH_SLIDES.pdf`).

Limits (say them before you're asked)

- Hybrid authentication was only measured against our own server.
- `0xFE20` is a custom extension. The production path is composite ML-DSA certificates.
- Keys are pinned instead of issued through a certificate authority.
- The bitmanip build needs Zba/Zbb.
- The key-exchange halves are still sequential inside OpenSSL.
- The 750 MHz setting isn't permanent on the board yet.
- Energy wasn't measured.

The whole story in one line: we made the board quantum-safe, found it was 30 % slower, traced the cause to the clock, the compiler and unused cores, fixed each in software, and ended up 31 % faster than the original legacy connection.

Source: TECHNICAL_WALKTHROUGH.md